ChatGPT

# Follow-Up Technical Vision: Optimizations, Deployment, and Security

## Introduction and Recap

In our initial briefing, we outlined a **web content filtering platform** that acts as a proxy to intercept web pages and filter out disallowed content before rendering to users. The goal of such a platform is to **block harmful or inappropriate content** from reaching end-users [1]. In the first phase, we described how an incoming HTTP request is fetched by the proxy, processed through an LLM-driven pipeline that builds a semantic knowledge graph of the page content and classification, and then reconstructed and delivered to the user with forbidden elements removed. This follow-up document builds on that foundation, focusing on **optimization strategies**, flexible **deployment options**, and key **security considerations** for the platform. Where relevant, we reference concepts from the previous document and maintain continuity with the same source material.

## Optimization Strategies for Performance

**1. Minimal Overhead in Monitoring Mode:** The platform supports a **"monitor-only" mode (Mode 1)** where pages are not actively filtered but merely logged/monitored. In this mode, the proxy fetches the content and immediately relays it to the user while asynchronously storing a copy of the HTML (e.g. in cloud storage like S3). This introduces only minimal latency overhead – essentially the time to save the page data – which is negligible compared to the network latency of fetching the page itself. Because the write to storage happens within the cloud data center, it should be on the order of milliseconds, much smaller than typical web request times. For early deployment, this saving can be done in-line (synchronously) for simplicity. If needed, further micro-optimizations (like offloading the save to an asynchronous worker) could shave off those few milliseconds, but initially keeping the flow simple reduces complexity. The key point is that **monitoring mode feels almost as fast as normal browsing**, since no heavy processing is done in the critical path.

**2. Active Filtering Mode and Caching: Mode 2** is the full **content filtering mode** where the system must analyze the page and possibly block or blur certain content in real-time (as described in the earlier document's pipeline). Here, performance is more challenging, but we can leverage caching and incremental processing to optimize it. Instead of treating each page as a completely new document, we break pages down into **segments** (or content blocks) and compute a unique **hash** for each segment of text. This way, if a segment has been seen and classified before, we can reuse that result without reprocessing it. In practice:

- **First visit to a page:** The HTML is parsed into a DOM tree and all textual nodes (and possibly meaningful DOM sections) are hashed. The system queries the cache (our stored knowledge graph database) to see if those hashes were seen before. For new content (hash misses), we invoke the LLM to classify it and add it to the knowledge graph with its classification label. Common, repetitive parts of the page (header, footer, menus, sidebars) will likely produce hashes that **repeat across many pages of the same site**, so the very first page visit to a site incurs most of the LLM processing cost, but subsequent pages benefit from reuse.

- **Exact repeat visits:** If the **exact same page content** is fetched again (e.g. user revisits or refreshes and nothing changed), all text segment hashes will be hits in cache. The proxy can then nearly instantaneously reconstruct the filtered page from cached results with no new LLM calls needed. Only the initial fetch and storage are done, making repeat view latency extremely low.

- **Partial overlap (incremental changes):** If a page has some new content but many sections unchanged (e.g. a news homepage updated with a new headline but other stories remain the same), the system only needs to process the **new hashes** that haven't been seen. Everything else can be looked up. This yields significant speed-ups. Effectively, we shift from caching whole pages to caching **fine-grained content segments**. Over time, as users browse multiple pages of the same domain, a large portion of each new page will hit in the cache.

This approach takes advantage of the fact that **web pages often have consistent templates or boilerplate content**. Research on duplicate detection and boilerplate removal supports this: many of the repeated "boilerplate" parts of pages (navigation menus, footers, common sidebars, etc.) share lots of overlapping tokens or shingles, whereas the unique article content has more novel tokens [2]. In other words, *"many of the shingles in the non-content portions of a webpage appear over and over again as part of the page boilerplate"* [2]. By identifying and caching these repetitive sections, the system avoids re-classifying the same boilerplate on every page.

**3. DOM Structure and Segmentation:** To maximize the above caching strategy, we consider the page's **DOM structure**. Rather than treating the HTML as a flat stream of text, we analyze it in logical sections (e.g. distinguishing the header, sidebar, main content, advertisements, etc.). The platform can maintain an **ontology or taxonomy of page sections** (with help from heuristic rules or even an LLM) to label different parts of the page. This opens up several optimization opportunities:

- We can prioritize certain sections for analysis (for example, the main article text might need stricter or more immediate filtering than a list of "related links" in the footer).
- If certain sections are known to rarely contain disallowed content (say, navigation menus are usually benign), we might handle those with simpler rules or less frequent checks.
- Segmenting also aids caching: we can compute a composite hash for large sections (like the entire header block). If the header's hash matches a cached version, we skip diving into its sub-elements at all. By choosing an optimal granularity (section-level vs element-level), we minimize the number of hashes/lookups needed.

There is a balance here: too fine-grained (hash every tiny `<span>`) could be overhead; too coarse (hash the whole page as one blob) misses the chance to reuse partial content. Our approach leans toward hashing **hierarchical blocks** – for example, first hash major sections (header, sidebar, main content), and if a section is new, then hash its sub-elements down to individual text nodes. By doing this, on many sites the top and side sections yield cache hits and we only drill down into the main content which is new.

It's worth noting that accurately identifying "main content" vs. boilerplate is a non-trivial task on complex websites. Simple heuristics (like "pick the largest `<div>` of text") often fail on modern layouts [3]. Advanced algorithms exist that use multiple heuristics or even ML/NLP to extract the primary content from webpages [3]. Our platform can leverage some of these techniques (possibly with the LLM itself assisting in tagging sections) to improve segmentation. By acknowledging this challenge (as **content extraction from web pages is a well-known problem** [3]), we plan to iteratively refine the section detection logic, starting with basic DOM parsing (e.g. using HTML tags, element ids/classes that often indicate menus, etc.) and later incorporating learned patterns for each site.

**4. Batch vs. Streaming LLM Calls:** Once we have identified the text segments that require classification, we face a decision on how to send them to the LLM for analysis. Two broad strategies are considered:

- **Single Batch Request:** Concatenate or structure all new text segments into one large prompt for the LLM, essentially asking it to classify all segments in one go. This ensures the LLM has full context and requires only one API call, but it might hit token limits for very large pages, and the latency will scale with the total size. There's also a risk that if the page is large, a single call could be slower (and more expensive) than needed, since it processes serially.

- **Parallelized Multiple Requests:** Break the classification task into multiple smaller prompts (for example, one prompt per paragraph or per small group of segments) and send them in parallel to multiple LLM instances or threads. This could drastically reduce wall-clock time, as many pieces are processed concurrently. The trade-off is some overhead per request (each prompt includes fixed tokens like instructions) and potentially higher total token usage cost. It also complicates recombining the results.

Our design will be flexible to experiment with both. We suspect an optimal middle-ground: e.g. splitting the page into N chunks that are each reasonably sized for the LLM's context window, and running N calls concurrently. This can reduce latency almost linearly with N (assuming the LLM provider allows that concurrency) at the cost of a slight increase in total tokens (due to repeated prompt overhead). We will benchmark configurations to find a sweet spot that minimizes user-visible latency without inflating cost too much. In essence, **we trade off compute cost for speed** – an important theme which we will also expose as a user-tunable setting (more on this below).

**5. Intelligent Chunking and Reconstruction:** Related to the above, when we split content for LLM analysis (whether by paragraphs or sections), we must ensure we can accurately **reconstruct the page** after classification. The knowledge graph stores each content node's classification and likely some placeholder or transformed text (e.g. a "[BLOCKED]" token for disallowed content). To reassemble the filtered HTML, we maintain the original DOM structure with references to the content nodes. We may introduce an explicit position index for each text node (for example, indicating its order within its parent element) so that even if we classify a whole paragraph as one chunk, we can map the result back to the exact spot in HTML. This way, we preserve the layout. The previous design already proposed storing enough context (parent-child relationships in the graph) to regenerate the page. We will extend that to handle grouped nodes. For example, if we send a full paragraph containing some bold or italic sub-elements as one chunk to the LLM, the returned classification might apply uniformly to that paragraph. We then would mark the entire paragraph node as allowed or blocked, rather than each sub-span individually, simplifying the replacement with a masked version if needed.

**6. Multi-Model and Cost/Quality Trade-offs:** Not all content classification needs the most powerful (and expensive) model. We intend to allow **configurable LLM backends** and even multiple models in tandem:

- **Choice of model:** A user (or admin) can choose whether to use a faster, cheaper model (which might be sufficient for simple content or when cost is a concern) or a more powerful model (like GPT-4) for higher accuracy on nuanced content. This **user-configurable performance vs. cost slider** empowers clients to decide what's more important for their use case (fastest response vs. most thorough filtering vs. lowest cost). The system will support various LLM APIs (OpenAI's models, open-source models, etc.) that can be swapped.

- **Ensemble for higher assurance:** In critical scenarios, the platform could send content to multiple models and **cross-verify** the classifications. For example, run two different models or run the same model twice and see if the results agree. If there's disagreement (one flags content as disallowed and another doesn't), a higher-level logic could resolve it (e.g. requiring a majority vote to block, or using the more conservative outcome). This ensemble approach would increase both latency and cost, so it would likely be an opt-in "strict mode." It could be useful for high-security environments where false negatives (letting bad content through) are unacceptable. The design is pluggable enough to accommodate this – essentially treating each model's output as a vote in a decision function.

- **Cost awareness:** We will incorporate mechanisms to estimate and monitor the token usage and response times for the chosen strategy. The system might automatically fall back to a simpler mode if the content size is too large (to avoid an extremely large single LLM call). Likewise, for batch processing of many segments, we might use smaller models for initial filtering and reserve big models for borderline cases (a form of two-tier filtering). All these optimizations ensure the solution can run **economically at scale** without sacrificing the core function.

**7. Prefetching and Cache Warming:** Another strategy borrowed from web performance best practices is **prefetching likely-needed content** to hide latency. Research shows that faster load times dramatically improve user experience (and even conversion rates), and one way to achieve faster loads is to fetch resources *before* the user explicitly navigates to them [4]. We can leverage this in a few ways:

- **Link prefetching:** When a user loads a page through our proxy, we can analyze that page (or use known patterns) to predict what page they might visit next (e.g. the top link on a news homepage, or a "next page" link in a multi-page article). The proxy could proactively fetch and process those linked pages in the background. Then, if the user does click one, it will load near-instantly because the content is already cached and pre-analyzed (a cache hit yields a very low Time to First Byte since the document request results in a local cache hit [5]).

- **Site crawling**: For known sites (especially those that our users frequent), we could run a background **crawler/agent** that periodically visits and processes popular pages, essentially keeping the cache "warm." For example, if many users read BBC News, our system can periodically fetch the latest articles from BBC's RSS feed or front page. This means the first user to request a new article benefits from the work already being done. This is akin to making our proxy a **smart caching layer with proactive population**, rather than a purely reactive filter.

- **Safety and efficiency:** We will do prefetching carefully to avoid wasting bandwidth on things the user never accesses. We'll use insights (perhaps an LLM can even help prioritize links that look most relevant) and ensure we don't prefetch on slow networks or for too many links at once [6] [7]. Additionally, prefetching will respect robots rules and not hammer websites excessively. It's an optimization to improve perceived performance for users at the slight cost of extra compute/bandwidth on our side.

Overall, these optimization strategies aim to **make the filtering as real-time as possible**, approaching the responsiveness of normal web browsing. Techniques like caching at the segment level, intelligent chunking, parallel LLM calls, and prefetching collectively can mitigate the inherent added work of content analysis. It's a recognition that the fastest request is the one you don't have to compute at all – so we cache and reuse everything we can, and try to move work out of the critical path via pre-processing or background tasks.

**8. Progressive Rendering (Async Filtering Mode):** A final idea under consideration is a **"fail-open" filtering mode** for certain use cases, where the page is initially delivered unfiltered and then the filter applies in real-time as results come in. Essentially, the proxy would stream the page to the user immediately (for minimal delay), *then* a few moments later (once the LLM classifications return) send an update (via a websocket or a small script on the page) to hide or remove content that was found disallowed. This is analogous to content being dynamically blurred out after the fact. It creates a risk that users might glimpse something before it's removed, so it's only suitable in environments where a brief exposure is acceptable (or where filtering is more about advising than strictly preventing viewing). This is somewhat similar to a **"fail open" configuration in traditional filters**, where if the filtering service is unavailable, traffic is allowed rather than blocked. (For example, Barracuda's web filter agent can be set to *Fail Open* so that if it cannot contact the filtering server, users are not blocked – they continue accessing the web unfiltered until the service restores [8].) In our case, *we would intentionally allow a brief fail-open period* to trade off absolute safety for performance. This mode isn't default, but it could be offered as an option for power users who prefer speed and are willing to tolerate that trade-off. It's a parallel to how some parental control systems might show content then retroactively log or hide it.

In summary, **optimization** in this platform centers on **caching, parallelism, and intelligent trade-offs**. We want to minimize redundant work (through hashing and reuse of previously seen content), utilize concurrency and precomputation (parallel LLM calls, prefetching next pages), and give configurable levers to balance cost, speed, and strictness of filtering.

## Flexible Deployment Strategies

One of the strengths of our approach is that it can be packaged to run in a variety of environments with minimal changes. At its core, the system requires a computing runtime to execute the proxy logic & LLM calls, and a storage layer for caching data. This can be deployed in the cloud in a multi-tenant fashion, or in isolation per customer, or even on-premises. Here we describe deployment options and how they align with security/privacy needs:

**1. Serverless Cloud Deployment (Multi-Tenant SaaS):** In the simplest form, we will deploy the proxy as a set of **serverless functions** (for example, AWS Lambda) behind an API Gateway. Each web request triggers a function that performs the fetching, analysis, and response assembly. Serverless infrastructure offers automatic scaling – functions will run in parallel as needed, and scale down to zero when idle – and removes the need to manage servers. It's cost-efficient and can handle spiky traffic patterns easily [9] [10]. The entire user base can share this infrastructure (multi-tenant). The cached content repository (e.g. an S3 bucket or a database) would also be shared. This has the advantage that if one user has already fetched and processed a given URL, another user's request for the same URL can directly use the cached result without reprocessing. In essence, the **filtering knowledge base is crowdsourced across tenants**, improving performance for everyone. This is an efficient model akin to a shared "learning" across the system.

However, multi-tenancy means multiple customers' data resides in the same logical system. While the content being filtered is typically public web data (not a user's private data), there may still be privacy concerns (e.g., one company might not be comfortable knowing that the system's cache has records of what another company's users browsed, even if they cannot see it). We mitigate that by strict **tenant data isolation** in software: each piece of cached data is tagged by which tenant/user it belongs to, and the application will ensure that one tenant cannot query or retrieve another's entries. This is analogous to multi-tenant SaaS models where all customers share the infrastructure but logically their data is separated (often by a tenant ID column, etc.) [11]. Robust isolation is critical – as one guide explains, a

multi-tenant system is like an apartment building: everyone shares the building but **each tenant's data should be isolated as if behind a locked door** [12]. We'll enforce that the proxy and storage only allow access to content for the requesting user's session.

**2. Dedicated Cloud Deployments (Single-Tenant):** For customers requiring stronger isolation (for compliance or internal policy reasons), we will offer dedicated deployments. This could mean running a separate set of Lambda functions (or separate function instances in isolated VPCs) and a dedicated storage partition for that client. Essentially, it's their own instance of the service, not shared with others. In terms of levels of isolation, this corresponds to a model where each tenant might even have their **own database or storage schema** [13] and potentially even separate compute clusters. It's more resource-intensive, but provides peace of mind that no data is commingled. Tenant isolation can be viewed as a spectrum: from fully isolated (every tenant on totally separate resources) to fully shared (everyone on one stack) [14]. We can accommodate anywhere along that spectrum. For example, one intermediate approach is to use a shared infrastructure but with **per-tenant encryption keys** so that even at the data storage level, one tenant's data cannot be decrypted by another [15]. In fact, we can encrypt each tenant's cached content at rest with a tenant-specific key; even if the storage is shared, this ensures privacy (this aligns with best practices where *"data encryption in multi-tenant systems is essential for privacy"* [16] and using distinct keys means a breach of one tenant's key doesn't expose others).

Dedicated deployments could be managed by us in our cloud but siloed for the client, or we could deploy into the **client's own cloud environment**. Modern infrastructure-as-code makes it feasible to ship a deployment package (e.g. a Terraform script or a Docker container with Helm charts for Kubernetes) that a client can run in their AWS/Azure account or on their Kubernetes cluster. The system's design (using stateless compute components and standard storage backends) is cloud-agnostic enough to run on AWS, Azure, GCP, or on-premises with **Kubernetes or even bare-metal**. Containers provide the portability to run the application anywhere consistently [17]. By containerizing the proxy service and using a portable database or filesystem interface for storage, we ensure that a client who wants to run it internally can do so with minimal fuss. This addresses scenarios where data sovereignty or ultra-low latency (keeping the server near users) is needed.

**3. On-Premises and Offline Use:** Some clients (e.g., a secure enterprise or government network) might require that absolutely **no data leaves their premises**. For these cases, the platform can be delivered as an on-prem appliance (probably a set of Docker containers or VMs). All components – the proxy, the storage DB, even the LLM if needed – would run locally in their network. The "offline" scenario (no Internet access except the sites being filtered) is also possible; the system just needs access to the web content and to whatever LLM model is used. If an internet-based LLM API is not permissible, the solution could integrate an offline model (for example, a smaller open-source language model running on local hardware or a private server). The flexibility of the design means we can plug in different LLM backends (including ones hosted by the customer). This ensures even environments with strict data control can benefit from the content filtering platform by **"bringing the filter to their data"** rather than sending their data out.

**4. Customer-Provided API Keys / Models:** In shared deployments, by default the LLM API calls (e.g. to OpenAI) would use our service's credentials. But we recognize some clients might have existing contracts or preferences (e.g., they have an OpenAI enterprise account with a certain data usage policy, or they prefer Azure OpenAI or another provider). We will allow customers to **provide their own API keys** or endpoints, which can be used by their instance of the service. For example, if a client loads our browser extension or configures the proxy, they could input their own OpenAI API key (which the software would then use for calls on their behalf). This way, any data sent to the LLM stays within the scope of their account with the LLM provider, and we as the filtering service do not retain those prompts or outputs beyond what's needed for filtering. It essentially **adds an extra layer of privacy** – from our

perspective, the content is never sent to a third-party except under the client's own agreement with that party. It also lets clients choose region-specific endpoints or models that align with their compliance needs.

In summary, the deployment model is very **flexible**: it can be a multi-tenant cloud service for convenience, but easily pivot to dedicated or on-prem deployments for privacy. Containers and serverless functions are complementary here – containers give us *portability* across environments [17], while serverless gives *easy scaling* in the cloud. We intend to support both as needed. This modular approach also means feature updates can be rolled out to the shared service and also packaged for on-prem customers with minimal changes.

Finally, it's worth noting from a business perspective: these deployment options can correspond to different pricing tiers (with multi-tenant SaaS being the most cost-effective, and fully isolated on-prem being a premium offering due to the added complexity). The architecture is designed to **reuse the same core codebase** across these scenarios, changing only configuration (for example, which storage to use, which keys, etc.).

## Security and Privacy Considerations

Security is paramount for a filtering platform, both in terms of the **integrity of the filtering itself** (ensuring no forbidden content slips through or malicious actors can tamper with it) and the **privacy of user data** (ensuring we don't introduce new risks by processing users' web traffic). We address these on multiple levels:

**1. Data Isolation and Tenant Security:** As discussed, multi-tenant deployments demand strong isolation. We will implement **tenant-specific access controls** in the data store so that even if multiple clients' data reside in the same physical database, they cannot see each other's records. Techniques include using separate partitions or schemas for each tenant, or at minimum a rigorous row-level filtering by tenant ID on every query [11]. In more sensitive setups, we opt for separate databases entirely [18] or even separate infrastructure. The principle is that **one customer's browsing data and results should never be accessible to another customer**. By default, our cloud service will **tag and encrypt data per tenant**: for instance, using a unique encryption key per tenant for data at rest so that even if the storage is shared, data remains unintelligible without the proper key [15]. If a malicious actor somehow got read access to the storage, they'd still need to breach the key management to get any usable information – an added layer of defense.

**2. Encryption and Secure Transmission:** All communication in the system will be encrypted. The proxy will communicate with clients over HTTPS (ensuring that the filtered content delivered to the user is encrypted in transit). Similarly, requests from the proxy to external websites will use HTTPS whenever the original site supports it (we won't downgrade any security). For data at rest, as mentioned, we use encryption (cloud storage encryption or application-level encryption with keys). The multi-tenant encryption approach was highlighted in the WorkOS guide: *"Data encryption in multi-tenant systems is essential for securing tenant data and ensuring privacy"* [16]. We abide by that through AWS KMS or similar services to manage keys (potentially with tenant-specific keys in dedicated deployments). In transit to LLM providers, we also use TLS and we will only integrate with providers who have strong data security measures. OpenAI's API for example uses HTTPS and offers options to not log or use data for training. We will document to customers that using the filtering service implies their page content is being sent to an AI API (unless they opt for an on-prem model). For the highest security needs, they can choose an on-prem LLM so that no data leaves their network.

**3. Safe Handling of Sensitive Content:** The irony of a filtering tool is that by processing potentially harmful or sensitive content, the tool itself becomes a holder of that content (even if temporarily). We plan to **minimize retention** of data. The cached pages and analysis results are stored primarily to improve performance and to provide user reporting, but we don't intend to keep raw logs longer than necessary. Possibly we'll implement a data retention policy: e.g. automatically purge cached content after X days if it hasn't been accessed, or store only aggregated info about it. We will also *sanitize logs* – for instance, if we log that "User X accessed URL Y and it was blocked for reason Z", we might not need to store the entire page contents indefinitely, just the classification and URL. For any stored content, encryption (as above) protects it at rest.

Furthermore, in early stages, we will warn users not to use the service for highly sensitive browsing (e.g. personal email, banking, proprietary internal sites) since those could contain private data that we don't want to inadvertently store. In fact, we can implement **domain whitelists/blacklists** to enforce that: e.g., by default the proxy might refuse or bypass itself for known sensitive domains (like banking sites, corporate intranets, etc.). That acts as a **circuit breaker** to reduce risk – those sites would load directly (or not at all) rather than through our filter until we're confident in our data handling for them. This is an extra precaution given the MVP nature of the project.

**4. Integrity of Filtering Decisions:** Security also means making sure the filter cannot be easily bypassed or tricked. Because we rely on an LLM for classification, one risk is if someone finds an adversarial input that the LLM misclassifies (thus the filter might let it through). To mitigate this, we'll continuously update our content classification criteria (possibly fine-tune the model or use multiple models as discussed) so that obvious bad content isn't missed. The knowledge graph approach also means we can retroactively apply improved classifications: if we discover that a certain phrase or content should have been blocked but wasn't, we can update the classification in the cache and the next time that content appears, it will be caught. We should also run **test suites** of sample pages to validate the filter (especially for high-risk categories like adult content, malicious scripts, etc.). Over time, incorporating a secondary rules-based scanner (for known patterns that should always be blocked) could supplement the LLM's decisions, providing defense-in-depth.

**5. Availability vs Security (Fail-Open vs Fail-Closed):** In a filtering system, **"fail-open"** means if the filter system fails for some reason, traffic is allowed through unfiltered; **"fail-closed"** means if the filter fails, all traffic is blocked (to avoid unfiltered access). Each approach has security implications. Fail-closed is safer (never allows bad content if the system is down), but can disrupt business (internet becomes unavailable if the filter service crashes or is unreachable). Fail-open keeps things running (users continue to access sites) but at the risk of unfiltered content. Many commercial products opt for fail-open to prioritize availability – for example, Barracuda's Web Security Gateway recommends fail-open so that users aren't completely cut off in case of outages [19]. Our design will allow the deployment owner to configure this behavior. By default, we might choose **fail-closed** during the initial rollout (to enforce that nothing slips by undetected), but in practice, because our proxy is inline, a true fail-closed means if our service is down, users can't browse at all. To be pragmatic, we will lean towards a hybrid: the proxy could detect if it's not able to get a classification in a timely manner and either temporarily fail-open (let the content through with a log entry) or degrade to monitor-mode. We will also have health checks – if the LLM API or our processing is failing, the system could automatically switch to a safe mode. These are operational considerations that we'll iron out with usage feedback.

**6. Tenant-Specific Keys and Credentials:** We touched on this, but it's worth emphasizing: when a deployment is isolated for a tenant, we can use keys such that *even we (as the service provider) don't have access* to certain data. For instance, if the client supplies their own OpenAI API key, that key can be stored in their environment (or an encrypted secret we can't read) – thus all prompts go directly to OpenAI under their account. Similarly, if we allow client-provided encryption keys for data at rest, only

the client can unlock the content (this would complicate some features like our ability to troubleshoot or audit, but it's an option for highly sensitive orgs). Using *tenant-specific encryption keys* is a recommended practice to ensure one compromised key doesn't affect others [15].

**7. Compliance and Auditability:** Enterprises will ask if the filtering platform complies with relevant regulations (GDPR, etc.). Because our system can potentially log user browsing (which could be personal data), we need to provide means to **export or delete user data on request**, honor do-not-track to an extent, and generally be transparent about what we store. We will implement admin dashboards where a customer can see the logs of blocked content for their users (fulfilling an audit purpose), and also have the ability to purge those logs. For GDPR, treating the browsing logs as personal data means we should delete data when a user leaves the organization or upon request. These policies will be established as we move from MVP to production.

**8. Hardening the Proxy:** The proxy itself should be secure against external attacks. It will be exposed to the internet (since it fetches arbitrary URLs), so we'll harden it by: - Running with least privilege (if using Lambda, the role has minimal permissions; if container, no unnecessary ports or root access). - Sanitizing any data we pass to other systems (to prevent injection attacks, though primarily we're the ones fetching – we must ensure that a malicious page can't, for example, exploit our HTML parser; using well-maintained libraries and sandboxing the DOM parsing/LLM calls is important). - The user-facing endpoint will have authentication (in a corporate setup, only authorized users or devices use the filter – e.g. the browser extension or network settings will include an API key or the user will log in). We don't want an open proxy that others can abuse. So, tying requests to user identity and rate-limiting misuse is on the roadmap.

In conclusion, our security strategy is **multi-faceted**: it addresses *privacy* (data isolation, encryption, customer control), *integrity* (reliable filtering results, not bypassable), and *availability* (failover modes, stable infrastructure). By offering deployment models that align with a client's security posture (shared vs dedicated vs on-prem) and implementing best practices for multi-tenant security, we ensure the content filtering platform can be trusted as part of the customer's secure web gateway. We will continue to revisit security as the project evolves, especially as we integrate third-party AI services – any such integration will be vetted for its own security (for example, ensuring the LLM service we use does not retain or leak the data it processes, per its policy).

# Conclusion

This technical vision outlined how we will **optimize performance** (through caching, parallel processing, and smart use of LLMs), how we can **deploy the solution flexibly** in different environments (from our cloud to on-premises) to meet customer needs, and how we will uphold **security and privacy** throughout. These strategies build upon the initial design of the content filtering platform, ensuring that as we move forward, the solution is not only effective at filtering unwanted content but also fast, scalable, and secure. By combining advanced techniques (like content segmentation and prefetching) with prudent architectural choices (like containerized deployments and strong tenant isolation), we aim to deliver a platform that users can rely on for a safer web experience without sacrificing performance or control. All the considerations above will guide the next implementation phases and help address stakeholder concerns around latency, cost, and security for this project, as described in the voice transcript and our ongoing design discussions.

**Sources:**


- Fortinet, *What is Content Filtering?* – definition of content filtering [1]

- Web.dev (Google), *Prefetch resources to speed up future navigations* – on prefetching for faster load times [4] [5]
- Pavlina Fragkou et al., *Duplicate detection accuracy with boilerplate removal* – noting repeated boilerplate content across pages [2]
- Amy Leong, *Web Content Extraction with Heuristics & NLP* – on challenges of extracting main content vs boilerplate [3]
- ParallelStaff Blog, *Serverless vs. Containers* – on portability of containers vs serverless vendor lock-in [17]
- WorkOS Blog, *Tenant isolation in multi-tenant systems* – on levels of isolation (shared vs separate resources) [14] and importance of encryption per tenant [16]
- WorkOS Blog – recommendation to use unique encryption keys per tenant for data security [15]
- Barracuda Networks, *Fail Open and Fail Closed Modes* – explaining fail-open (filter disabled to not block traffic) [8]
- Albert Lee, *Craw4AI content processing pipeline* – example pipeline with content filtering, chunking, etc., similar to our design [20]

---


[1] What is Content Filtering? Definition and Types of Content Filters | Fortinet
https://www.fortinet.com/resources/cyberglossary/content-filtering

[2] shows how duplication detection accuracy, in terms of AU-ROC (on the... | Download Scientific Diagram
https://www.researchgate.net/figure/shows-how-duplication-detection-accuracy-in-terms-of-AU-ROC-on-the-Y-axis-varies-as_fig6_258240101

[3] Web Content Extraction with Heuristics & NLP | by Amy Leong | . | Medium
https://medium.com/aimonks/web-content-extraction-with-heuristics-nlp-8e901875c8ed

[4] [5] [6] [7] Prefetch resources to speed up future navigations | Articles | web.dev
https://web.dev/articles/link-prefetch

[8] [19] Fail Open and Fail Closed Modes with the Barracuda WSA | Barracuda Campus
https://campus.barracuda.com/product/websecurityagent/doc/172491428/fail-open-and-fail-closed-modes-with-the-barracuda-wsa/

[9] [10] [17] Serverless vs. Containers | Choosing A Cloud Strategy
https://parallelstaff.com/serverless-vs-containers/

[11] [12] [13] [14] [15] [16] [18] Tenant isolation in multi-tenant systems: What you need to know — WorkOS
https://workos.com/blog/tenant-isolation-in-multi-tenant-systems

[20] Webscraping in the AI Era: Craw4AI framework coding examination | by Albert Lee | Medium
https://medium.com/@albertlee_22659/webscraping-in-the-ai-era-craw4ai-framework-coding-examination-6a23f7b386d1